

Documentación Frontend (React)

Plataforma de Aprendizaje de Lenguaje C

Repositorio: plataforma-aprendizaje-c

Enero 2026

Índice

1. Resumen ejecutivo	2
2. Estructura del proyecto (frontend)	2
3. Tecnologías y librerías	2
3.1. Base path (deploy bajo /react)	2
4. Cómo se sirve el frontend	3
5. Ruteo del frontend (React Router)	3
5.1. Rutas públicas	3
5.2. Rutas protegidas	3
6. Sesión y autenticación	3
6.1. SessionProvider	3
6.2. RequireAuth	4
7. Layout y componentes repetidos	4
7.1. Shell	4
7.2. HeaderUsuario	4
7.3. Footer	4
8. Datos: Índice de módulos	4
8.1. Formato	5
8.2. Generación del índice	5
9. Página de Cursos (Hub)	5
9.1. Progreso del usuario	5
10. Viewer de contenido y por qué usa iframe	5
11. CSS y estilos	6
11.1. Dónde se editan estilos del React	6
11.2. Importante: estilos dentro del contenido (módulos)	6

12.Build, despliegue y comandos	6
12.1. Desarrollo	6
12.2. Producción	6
12.3. Regenerar índice de módulos	7
13.Problemas comunes y diagnóstico	7
13.1. Me redirige a login aunque estoy logueado	7
13.2. No aparece la lista de módulos	7
13.3. El contenido en iframe no carga	7
14.Extensiones futuras (sugeridas)	7

1. Resumen ejecutivo

Este documento describe el **frontend** de la plataforma: arquitectura, ruteo, sesión/autenticación, componentes y estilos.

El objetivo del frontend es:

- Proveer una experiencia SPA moderna construida en **React**.
- Reutilizar componentes comunes (**Header de usuario** y **Footer**) en toda la aplicación.
- Integrar el contenido existente (más de 100 HTML) sin romper funcionalidades interactivas como las **preguntas**.

2. Estructura del proyecto (frontend)

La parte React se encuentra en:

Proyecto de info/react-spa/

Archivos principales:

- `src/main.jsx`: entrada de React, monta Router y SessionProvider.
- `src/App.jsx`: definición de rutas.
- `src/index.css`: estilos globales (layout, hub, viewer, etc.).
- `src/components/`: componentes reutilizables (Shell, HeaderUsuario, Footer, etc.).
- `src/pages/`: pantallas (Landing, Login, Cursos, Viewer, etc.).
- `src/context/SessionContext.jsx`: estado global de sesión (usuario logueado).

3. Tecnologías y librerías

- **React**: UI declarativa.
- **React Router**: ruteo SPA con BrowserRouter.
- **Vite**: bundler y tooling de desarrollo/build.

3.1. Base path (deploy bajo /react)

El frontend está construido para vivir bajo el prefijo `/react`.

- Vite: `base: '/react/'` (ver `vite.config.js`).
- React Router: `basename="/react"` (ver `src/main.jsx`).

4. Cómo se sirve el frontend

Aunque este documento es del frontend, es importante entender la integración:

- Express sirve el build estático de React desde `react-spa/dist` en la ruta `/react`.
- Se configura un **fallback** para que `/react/...` devuelva `index.html` y React Router resuelva la ruta.
- Rutas antiguas HTML (por ejemplo `/login`, `/registro`, `/cursos-selec.html`) se re-dirigen hacia rutas React equivalentes.

5. Ruteo del frontend (React Router)

Las rutas del frontend se definen en `src/App.jsx`.

5.1. Rutas públicas

Ruta	Pantalla
<code>/</code>	Landing (entrada pública)
<code>/login</code>	Login (POST <code>/login</code>)
<code>/registro</code>	Registro (POST <code>/register</code>)
<code>/forgot-password</code>	Recuperar (POST <code>/forgot-password</code>)
<code>/restore-password</code>	Restablecer (POST <code>/restore-password</code> , usa query token)

5.2. Rutas protegidas

Estas rutas requieren sesión (controladas por `RequireAuth`):

Ruta	Pantalla
<code>/cursos</code>	Hub de cursos: lista de módulos y lecciones
<code>/m/:modIndex/:fileIndex</code>	Índice de contenido con anterior/siguiente
<code>/inicio</code>	Inicio (placeholder/migración mínima)
<code>/admin</code>	Admin (placeholder/migración mínima)
<code>/configuracion</code>	Configuración usuario (placeholder/migración mínima)

6. Sesión y autenticación

El frontend usa cookies de sesión del backend (`express-session`).

6.1. SessionProvider

`src/context/SessionContext.jsx` define un contexto global que:

- Llama a GET `/search-user` para obtener el usuario.
- Mantiene `status: loading | authenticated | guest`.

- Expone `refresh()` para revalidar sesión.
- Expone `logout()` (POST `/logout` + `refresh`).

Puntos clave:

- Todas las llamadas usan `credentials: 'include'` para enviar cookies.
- Cuando el usuario inicia sesión desde Login, se ejecuta `refresh()`.

6.2. RequireAuth

`src/components/RequireAuth.jsx` bloquea rutas protegidas:

- Si `status` es `loading`: muestra un estado de carga.
- Si es `guest`: redirige a `/login?from=...`
- Si es `authenticated`: renderiza el contenido.

7. Layout y componentes repetidos

7.1. Shell

`src/components/Shell.jsx` es el layout principal:

- Inserta **HeaderUsuario** al inicio.
- Renderiza el **contenido de la página**.
- Inserta **Footer** al final.

7.2. HeaderUsuario

`src/components/HeaderUsuario.jsx` muestra:

- Nombre de usuario cuando está autenticado.
- Botón de cerrar sesión que llama `logout()` y vuelve al inicio.
- Links de navegación (Cursos, Configuración).

7.3. Footer

`src/components/Footer.jsx` contiene estructura de pie de página (logo C, enlaces rápidos, copyright).

8. Datos: Índice de módulos

El hub de cursos consume un **índice** de módulos generado desde los HTML existentes.

8.1. Formato

Se lee desde:

/js-spa/data/modulosIndex.json

Estructura esperada:

```
[
  {
    "modulo": "modulo-1-Fundamentos-Introduccion",
    "archivos": [
      { "nombre": "100-Introduccion...html", "ruta": "/Modulos/.../100-...html"
    }
  ]
}
```

8.2. Generación del índice

El script `scripts/generateModulosIndex.js` recorre:
`extttproyecto/dist/protected_html/Modulos`
y genera:

- `js-spa/data/modulosIndex.js` (compatibilidad con SPA antiguo)
- `js-spa/data/modulosIndex.json` (consumido por React)

9. Página de Cursos (Hub)

Implementada en `src/pages/CursosSelec.jsx`.
Flujo:

- Usa `useModulosIndex()` para cargar el índice.
- Muestra tarjetas de módulo y links a lecciones.
- Incluye barra superior de progreso (`TopProgressBar`).

9.1. Progreso del usuario

Actualmente el progreso se obtiene desde `useUserProgress()` que llama a una API futura `/api/progress`. Si no existe, se muestra 0% sin romper la UI.

10. Viewer de contenido y por qué usa iframe

La mayor parte del contenido son documentos HTML completos (con head, scripts y CSS propio). Inyectarlos como `innerHTML` en una SPA suele romper:

- ejecución de scripts del documento,
- carga de estilos declarados en `<head>`,

- selectores globales que asumen documento completo,
- funcionalidades interactivas (por ejemplo: páginas de preguntas).

Por eso `src/pages/ModuloViewer.jsx` renderiza:

- `<iframe src=file.ruta .../>`
- Navegación **Anterior/Siguiente** calculada desde un arreglo plano (flat) de lecciones.

Esto preserva el comportamiento original del HTML.

11. CSS y estilos

11.1. Dónde se editan estilos del React

El archivo principal es `src/index.css`. Allí están:

- Estilos del hub (`.hub-*`).
- Estilos del viewer (`.viewer-*`).
- Estilos de navbar cards (`.navbar-*`).
- Estilos base (tipografía, body, links, etc.).

11.2. Importante: estilos dentro del contenido (módulos)

Como el contenido se carga dentro de un `iframe`, los estilos del React **no** afectan el interior del contenido.

Para modificar el look de los módulos, debes editar los HTML/CSS que esos HTML referencian, o bien crear una estrategia común (por ejemplo: enlazar un CSS global en todos los HTML).

12. Build, despliegue y comandos

12.1. Desarrollo

- Backend: `cd server` y `npm run dev`
- Frontend React (opcional): `cd react-spa` y `npm run dev`

12.2. Producción

- `cd react-spa`
- `npm run build`
- Express servirá `react-spa/dist` bajo `/react`

12.3. Regenerar índice de módulos

- `node scripts/generateModulosIndex.js`

13. Problemas comunes y diagnóstico

13.1. Me redirige a login aunque estoy logueado

- Verifica que el backend esté corriendo en el mismo dominio (`localhost:8080`).
- Revisa en DevTools que la cookie de sesión exista y se envíe.
- Confirma que `GET /search-user` responda `success: true`.

13.2. No aparece la lista de módulos

- Confirma que existe `js-spa/data/modulosIndex.json`.
- Ejecuta el script generador y recarga.

13.3. El contenido en iframe no carga

- Revisa que la ruta `/Modulos/...` exista y el backend la sirva.
- Si el backend protege la ruta, asegúrate de estar autenticado.

14. Extensiones futuras (sugeridas)

- Implementar `/api/progress` real (por usuario) para el `TopProgressBar`.
- Migrar gradualmente `/inicio`, `/admin`, `/configuracion` a vistas completas.
- Agregar un `vite proxy` para desarrollar React contra el backend en modo `npm run dev` sin problemas de cookies.